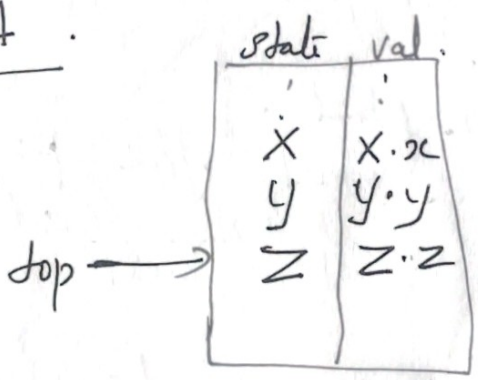
* When performed in depth-first order, the actions in fig. 5.20 print the o/p 95-2+.

Bottom-up Evaluation of S-attributed Definitions

* Synthesized attributes can be evaluated by a bottom-up parser as the input is being parsed.
* The parser can keep the values of the synthesized attributes associated with the grammar symbols in its stack.
* Whenever a reduction is made, the values of the new synthesized attributes are computed from the attributes appearing on the stack for the grammar symbols on the right side of the reducing production.
* A translator for an S-attributed definition can be implemented with the help of an LR parser generator.
* A bottom up parser uses a stack to hold information about subtrees that have been parsed.

* Here the stack is implemented by a pair of arrays state and val.
* If the ith state symbol is A, then val[i] will hold the value of the attribute associated with the parse tree node corresponding to A.



$A \rightarrow XYZ$

$A.a \rightarrow X.x, Y.y, Z.z$

Fig. Parser stack with a field for synthesized attributes.

* The current top of the stack is indicated by the pointer top.
* Consider the semantic rule $A.a = f(X.x, Y.y, Z.z)$ is associated with the production $A \rightarrow XYZ$, Before XYZ is reduced to A, the value of the attribute Z.z is in val[top], that of Y.y in val[top-1] and that of X.x in val[top-2].
* If a symbol has no attribute, then the corresponding entry in the val array is undefined.
* After the reduction, top is decremented by 2, the state covering A is put in state[top].

(ie, where X was) and the value of the synthesized attribute A.a is put in val[top].

* ~~The code fragment~~ When a production with r symbols on the right side is reduced, the value of ntop is set to top − r + 1. After each code fragment is executed, top is set to ntop.

top − top of the stack.
ntop − top of the stack after popping right hand side of the production.

Consider, production	Code fragment
$L \rightarrow En$	print (val[top])
$E \rightarrow E_1 + T$	val[ntop] = val[top−2] + val[top]
$E \rightarrow T$	val[ntop] = val[top]
$T \rightarrow T_1 * F$	val[ntop] = val[top−2] × val[top]
$T \rightarrow F$	val[ntop] = val[top]
$F \rightarrow (E)$	val[ntop] = val[top−1]
$F \rightarrow digit$	val[ntop] = val[top]

Fig - Implementation of a desk calculator with an LR parser.

Input	state	val	Production used
3*5+4n	—	—	
*5+4n	3	3	
*5+4n	F	3	F → digit.
*5+4n	T	3	T → F
5+4n	T*	3 —	
+4n	T*5	3 — 5	
+4n	T*F	3 - 5	F → digit
+4n	T	15	T → T*F
+4n	E	15	E → T
4n	E+	15-	
n	E+4	15-4	
n	E+F	15-4	F → digit
n	E+T	15-4	T → F
n	E	19	E → E+T
	En	19	
	L	19	L → En.

Fig. Moves made by translator on input ~~stream~~ 3*5+4n.

Top-down translation

* L-attributed definitions will be implemented during predictive parsing.

Eliminating left recursion from a translation scheme

Consider,

$E \to E_1 + T$	$\{E.val = E_1.val + T.val\}$
$E \to E_1 - T$	$\{E.val = E_1.val - T.val\}$
$E \to T$	$\{E.val = T.val\}$
$T \to (E)$	$\{T.val = E.val\}$
$T \to num$	$\{T.val = num.val\}$

Fig. Translation scheme with left recursive grammar.

$E \to E+T \mid E-T \mid T$

$E \to TE'$

$E' \to +TE' \mid -TE' \mid \varepsilon$

$E \to T\ R$	$\{R.i = T.val\}$ $\{E.val = R.s\}$
$R \to +\ T\ R_1$	$\{R_1.i = R.i + T.val\}$ $\{R.s = R_1.s\}$
$R \to -\ T\ R_1$	$\{R_1.i = R.i - T.val\}$ $\{R.s = R_1.s\}$
$R \to \varepsilon$	$\{R.s = R.i\}$
$T \to (E)$	$\{T.val = E.val\}$
$T \to num$	$\{T.val = num.val$

Fig. Translation scheme without left recursion.

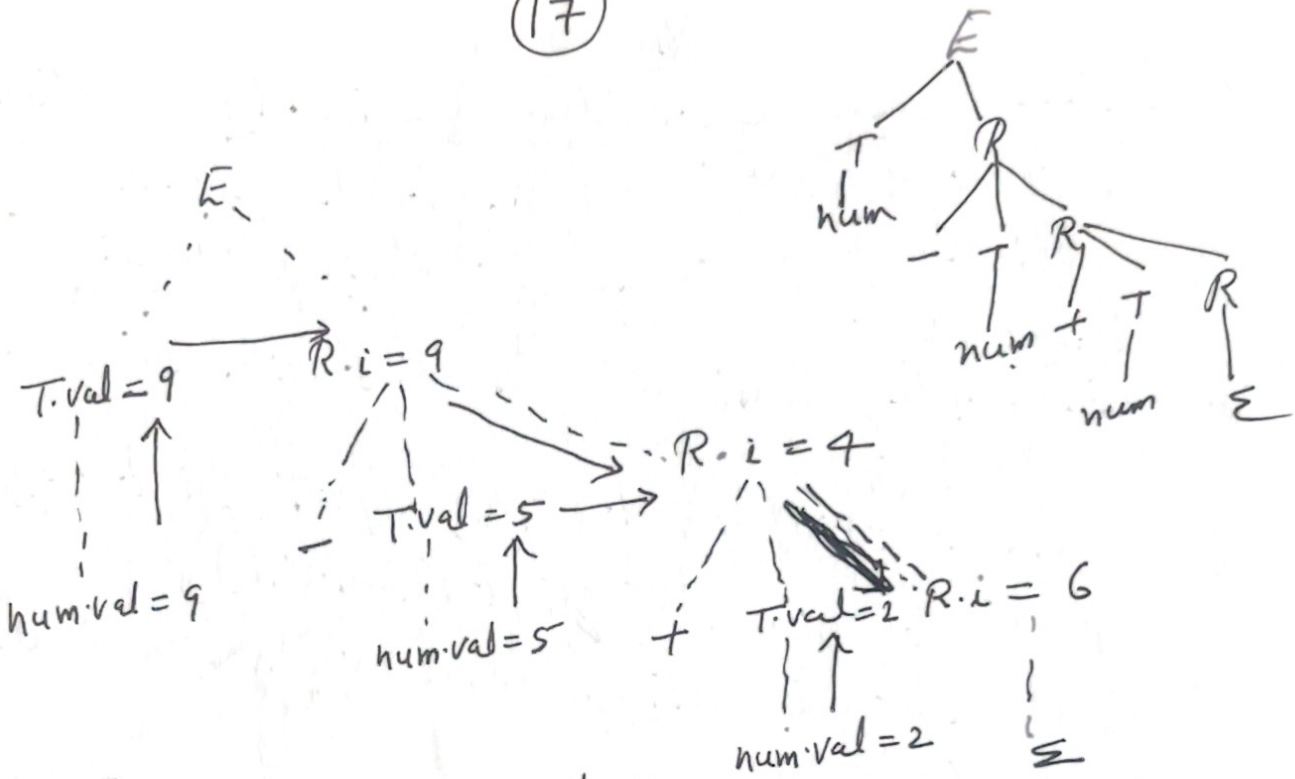


Fig. Evaluation of the expression 9-5+2

Bottom-up evaluation of inherited attributes

* Here a method to implement L-attributed definitions in the framework of bottom-up parsing.

Removing Embedding actions from translation Scheme

* Inherited attributes can be handled bottom-up by considering a transformation that makes all embedded actions in a translation scheme occur at the right ends of their productions.
* The transformation inserts new marker nonterminals generating ε into the base grammar.

* Replace each embedded action by a distinct marker nonterminal M and attach the action to the end of the production $M \rightarrow \varepsilon$

* Example, Consider the translation scheme,

$E \rightarrow TR$

$R \rightarrow +T \{ print('+') \} R \mid -T \{ print('-') \} R \mid \varepsilon$

$T \rightarrow num \{ print(num.val) \}$

is transformed using marker nonterminals M and N in to

$E \rightarrow TR$

$R \rightarrow +TMR \mid -TNR \mid \varepsilon$

$T \rightarrow num \{ print(num.val) \}$

$M \rightarrow \varepsilon \{ print('+') \}$

$N \rightarrow \varepsilon \{ print('-') \}$

* The grammars in the two translation scheme ~~terminate~~ ~~productions~~ accept exactly the same language and by drawing a parse tree with extra nodes for the actions, and can be shown that the actions are performed in the same order.

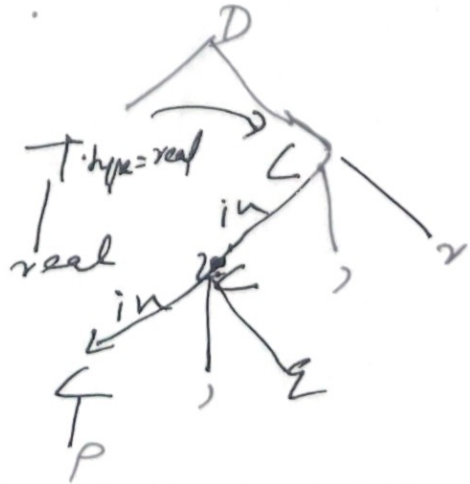
Inheriting attributes on the parser stack

- A bottom up parser reduces the right side of production $A \to XY$ by removing X and Y from the top of the parser stack and replacing them by A.
- Suppose X has a synthesized attribute $X.s$, kept along with X on the parser stack.
- Since the value of $X.s$ is already on the parser stack before any reductions takes place in the subtree below Y, this value can be inherited by Y.
- That is if inherited attribute $Y.i$ is defined by the copy rule $Y.i = X.s$, then the value $X.s$ can be used where $Y.i$ is called for.

$A \to XY$
$X.s \quad Y.i$

- Copy rule play an important role in the evaluation of inherited attributes during bottom-up parsing.

Consider real p, q, r



Given,

production	Semantic Rule
$D \to T\ L$	$\{L.in = T.type\}$
$T \to int$	$\{T.type = integer\}$
$T \to real$	$\{T.type = real\}$
$L \to L_1, id$	$\{L_1.in = L.in\}$ $\{addtype(id.entry, L.in\}$
$L \to id$	$\{addtype(id.entry, L.in\}$

Input	stack	production used
real p, q, r	real	
p, q, r	T	$T \to real$
p, q, r	Tp	$L \to id$
, q, r	TL	
, q, r	TL,	
q, r	TL, q	
, r	TL	$L \to L, id$
, r	TL,	
r	TL, r	
	TL	$L \to L, id$
	D	$D \to TL$

* When the production $L \rightarrow id$ is applied, id.entry is on top of the val stack and T.type is just below it. Hence addtype(val[top], val[top-1]) is equivalent to addtype (id.entry, T.type)

* Since the right side of the production $L \rightarrow L, id$ has three symbols, T.type appears in val[top-3] when this reduction takes place.

* The copy rules involving L.in are eliminated because the value of T.type in the stack is used instead.

Production	Code fragment
$D \rightarrow TL$	val[ntop] = integer
$T \rightarrow int$	val[ntop] = real
$T \rightarrow real$	addtype (val[top], val[top-3])
$L \rightarrow L, id$	addtype (val[top], val[top-1])
$L \rightarrow id$	

Fig. The value of T.type is used in place of L.in.